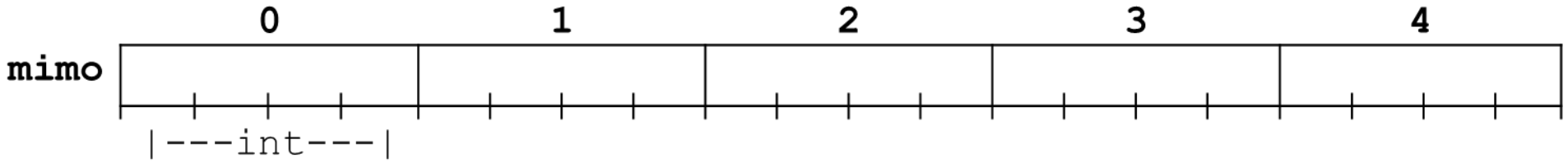
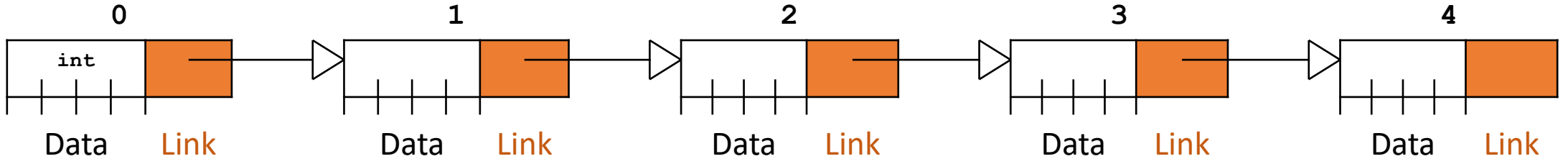
LINKED LIST

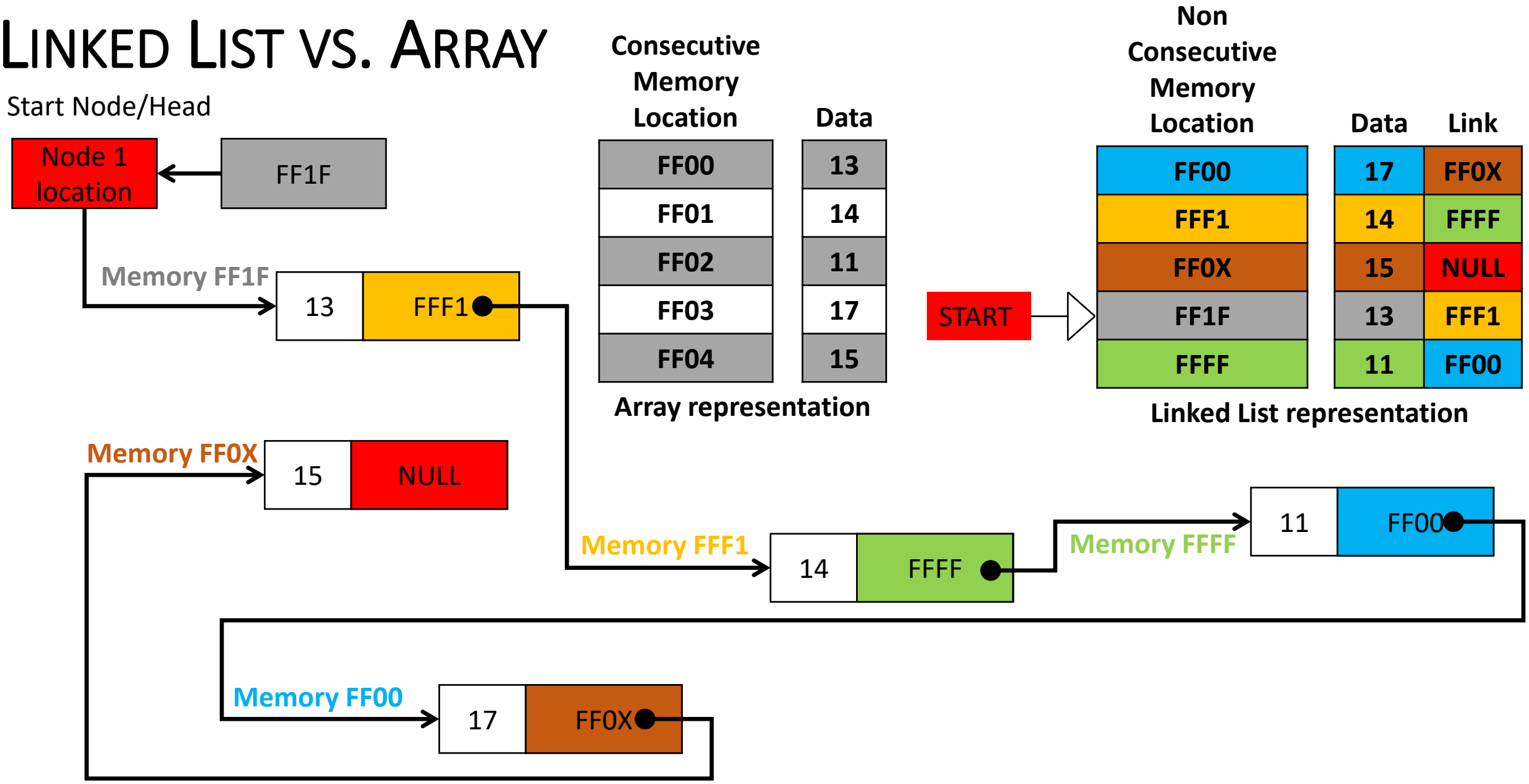
- Linked list is a data structure consisting of a group of memory space which together represent a list i.e. a sequence of data.
- A sequence of data can also be represented as an array. But in an array, data are stored consecutively in the memory.
- For example, an array to contain `5` integer values of type `int` called `mimo` could be represented like this:



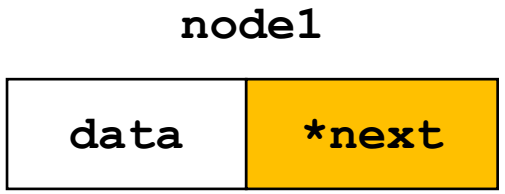
- A linked list is also a sequence of data. But in a linked list the data are not stored consecutively in the memory.
- Each data is stored in a separate memory space/block (called cell/node).
- Each memory block contains the data along with link/location/address to the memory location for the next data in the list.

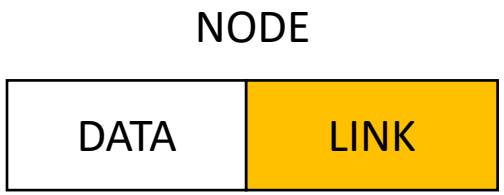


LINKED LIST VS. ARRAY



REPRESENTATION – NODE



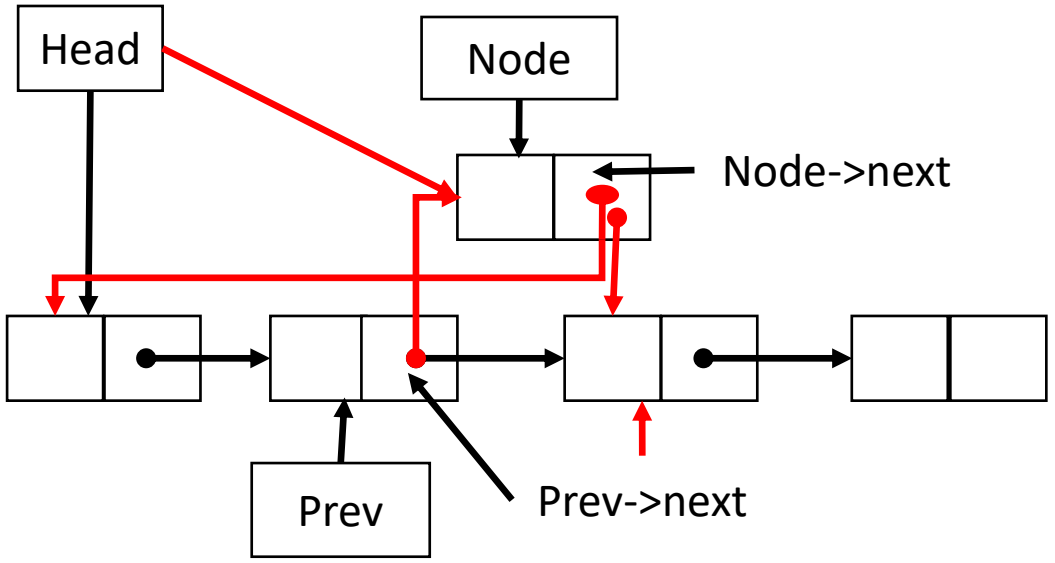


Representation of a NODE in C/C++

```
struct ListNode{
        int data;
        ListNode *next;
};
ListNode node1;
```

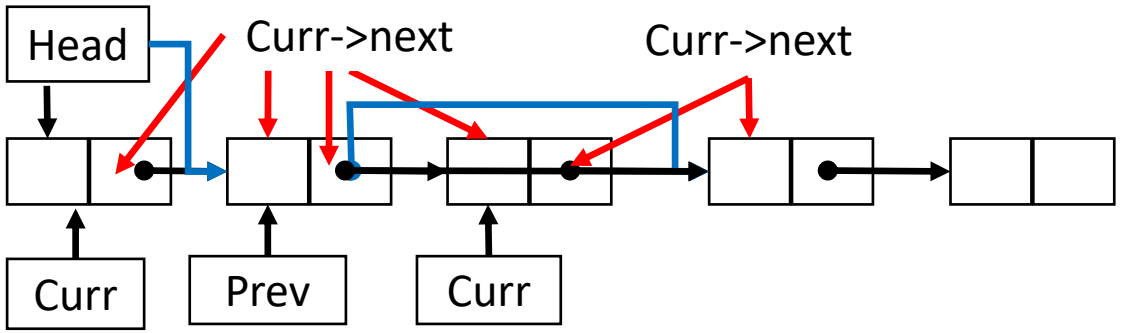
- Each NODE of the list contains –
  - The DATA item
  - A LINK/POINTER to store the next NODE's address.

LINKED LIST - CODING




```
// Inserting node after Prev Node
void InsertNode( ListNode *Prev, ListNode *Node){
if( Prev == NULL ){
        // inserting a node as head
   Node->next = Head;
   Head = Node;
   }else{
   Node->next = Prev->next;
   Prev->next = Node;
   }
}
```

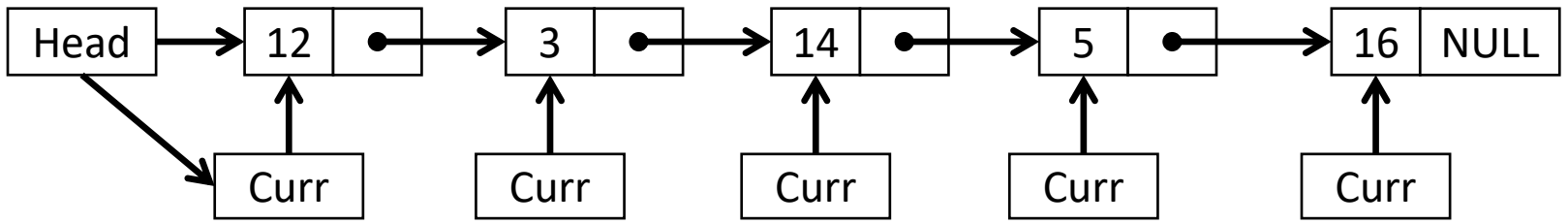
LINKED LIST - CODING




```
// Deleting node after Prev node
void DeleteNode( ListNode *Prev ){
 ListNode *Curr;
  if(Prev == NULL){
        // Deleting the head node
    Curr = Head;
    Head = Curr->next;
   }else{
    Curr = Prev->next;
    Prev->next = Curr->next;
   }
  delete Curr;
 }
```

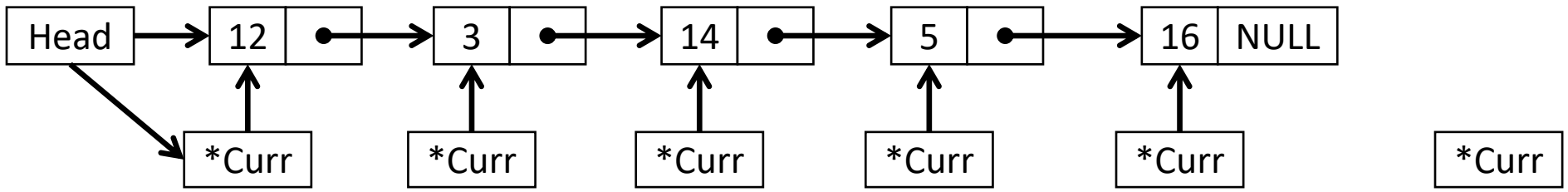
LINKED LIST - CODING

```
//Searching a data and giving the node address in *Curr and previous node address in *Prev
void SearchData(int data ){
  ListNode *Curr = Head;
 while( Curr != NULL ){
      if( Curr->data == data )      break;
      Curr = Curr->next;
  }
}
```

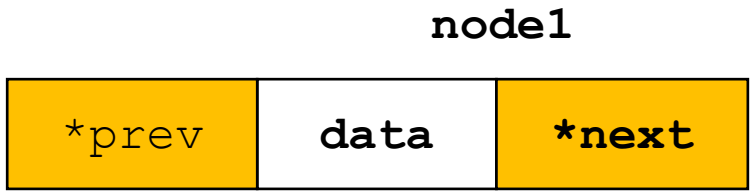


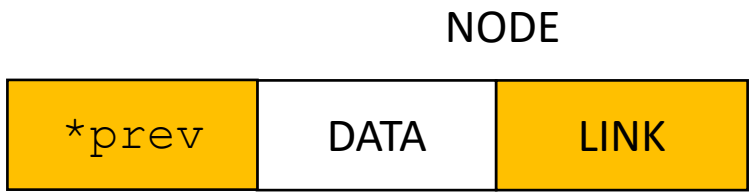
LINKED LIST - CODING

```
// Showing All the data sequentially from Head
void ShowList( void ){
     ListNode *Curr = Head;
        cout << endl;
     while( Curr != NULL ){
           cout << Curr->data << " ";
           Curr = Curr->next;
        }
        cout << endl;
}
```



DOUBLY LINKED LIST REPRESENTATION – NODE





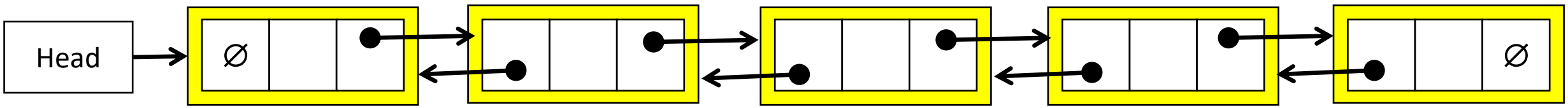
Representation of a NODE in C/C++

```
struct ListNode{
        int data;
        ListNode *prev;
        ListNode *next;
};
ListNode node1;
```

- Each NODE of the list contains –
  - The DATA item
  - A LINK/POINTER to store the previous NODE's address
  - A LINK/POINTER to store the next NODE's address

Doubly Linked List

- The *prev link of the head node and the *next link of the last node is NULL.
- While creating a list, *prev must be assigned to previous created node.
- InsertNode and DeleteNode parameters will be current node (*previous node in single linked list*). Because, current node contains both previous and next node address.
- In Searching, one parameter is sufficient to get both *prev and *next values.



DOUBLY LINKED LIST

- Insertion
- Deletion
- Search